Федеральное государственное образовательное бюджетное
учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики» (ФГОБУ ВО «СибГУТИ»)

# Основы систем мобильной связи

## Лабораторная работа 2

Выполнил: Клишин Владиммир Евгеньевич ИКС - 431

Новосибирск 2026

**Цель работы:**
Получить представление о том, как проектируется покрытие сетей мобильной связи и, научиться рассчитывать радиус действия (радиопокрытие) отдельных базовых станций БС (сот).

## 1) Выполните расчет бюджета восходящего канала, используя входные данные и определите уровень максимально допустимых потерь сигнала MAPL_UL.

Код:

```
import numpy as np
import matplotlib.pyplot as plt

TxPowerBS = 46
TxPowerUE = 24
AntGainBS = 21
PenetrationM = 15
MIMOGain = 3
IM = 1
FeederLoss = 2
NoiseFigureBS = 2.4
NoiseFigureUE = 6
ThermalNoiseUL = -174 + 10 * np.log10(10e6)
ThermalNoiseDL = -174 + 10 * np.log10(20e6)
ReqiredSINR_UL = 4
ReqiredSINR_DL = 2

RxSensBS = NoiseFigureBS + ThermalNoiseUL + ReqiredSINR_UL
RxSensUE = NoiseFigureUE + ThermalNoiseDL + ReqiredSINR_DL

MAPL_UL = (TxPowerUE - FeederLoss + AntGainBS + MIMOGain - IM - PenetrationM) - RxSensBS
MAPL_DL = (TxPowerBS - FeederLoss + AntGainBS + MIMOGain - IM - PenetrationM) - RxSensUE
```

## 2) Выполните расчет бюджета нисходящего канала, используя входные данные и определите уровень максимально допустимых потерь сигнала MAPL_DL.

Код:

```
import numpy as np
import matplotlib.pyplot as plt

TxPowerBS = 46
TxPowerUE = 24
```

```
AntGainBS = 21
PenetrationM = 15
MIMOGain = 3
IM = 1
FeederLoss = 2
NoiseFigureBS = 2.4
NoiseFigureUE = 6
ThermalNoiseUL = -174 + 10 * np.log10(10e6)
ThermalNoiseDL = -174 + 10 * np.log10(20e6)
ReqiredSINR_UL = 4
ReqiredSINR_DL = 2

RxSensBS = NoiseFigureBS + ThermalNoiseUL + ReqiredSINR_UL
RxSensUE = NoiseFigureUE + ThermalNoiseDL + ReqiredSINR_DL

MAPL_UL = (TxPowerUE - FeederLoss + AntGainBS + MIMOGain - IM - PenetrationM) - RxSensBS
MAPL_DL = (TxPowerBS - FeederLoss + AntGainBS + MIMOGain - IM - PenetrationM) - RxSensUE

print(f"MAPL_UL = {MAPL_UL} дБ")
print(f"MAPL_DL = {MAPL_DL} дБ")
```

Вывод:

```
MAPL_UL = 127.6 дБ
MAPL_DL = 144.9897000433602 дБ
```

**3)Постройте зависимость величины входных потерь радиосигнала от расстояния между приемником и передатчиком по всем трем описанным в п.2.2**
**моделям. Выберите нужную модель для заданных условий.**

Код:

```
import numpy as np
import matplotlib.pyplot as plt

TxPowerBS = 46
TxPowerUE = 24
AntGainBS = 21
PenetrationM = 15
MIMOGain = 3
IM = 1
FeederLoss = 2
NoiseFigureBS = 2.4
NoiseFigureUE = 6
ThermalNoiseUL = -174 + 10 * np.log10(10e6)
ThermalNoiseDL = -174 + 10 * np.log10(20e6)
ReqiredSINR_UL = 4
```

```
ReqiredSINR_DL = 2

RxSensBS = NoiseFigureBS + ThermalNoiseUL + ReqiredSINR_UL
RxSensUE = NoiseFigureUE + ThermalNoiseDL + ReqiredSINR_DL

MAPL_UL = (TxPowerUE - FeederLoss + AntGainBS + MIMOGain - IM - PenetrationM) - RxSensBS
MAPL_DL = (TxPowerBS - FeederLoss + AntGainBS + MIMOGain - IM - PenetrationM) - RxSensUE

print(f"MAPL_UL = {MAPL_UL} дБ")
print(f"MAPL_DL = {MAPL_DL} дБ")

def UMiNLOS(d_m):
PL = 26 * np.log10(1.8) + 22.7 + 36.7 * np.log10(d_m)
return PL

def COST231_Hata(d_km):
h_bs=30
h_ms=1.5
f_mhz=1800
A = 46.3
B = 33.9
a_hms = (1.1 * np.log10(f_mhz)) * h_ms - (1.56 * np.log10(f_mhz) - 0.8)
L_clutter = 0
s = 44.9 - 6.55 * np.log10(f_mhz)
PL = A + B * np.log10(f_mhz) - 13.82 * np.log10(h_bs) - a_hms + s * np.log10(d_km) + L_clutter
return PL

def Walfish_Ikegami_LOS(d_km):
f_mhz=1800
PL = 42.6 + 20 * np.log10(f_mhz) + 26 * np.log10(d_km)
return PL

d_m = np.linspace(1, 200, 1000)
PL_UMi = UMiNLOS(d_m)

d_km = np.linspace(0.1, 30, 1000)
PL_Hata = COST231_Hata(d_km)
PL_Walfish = Walfish_Ikegami_LOS(d_km)

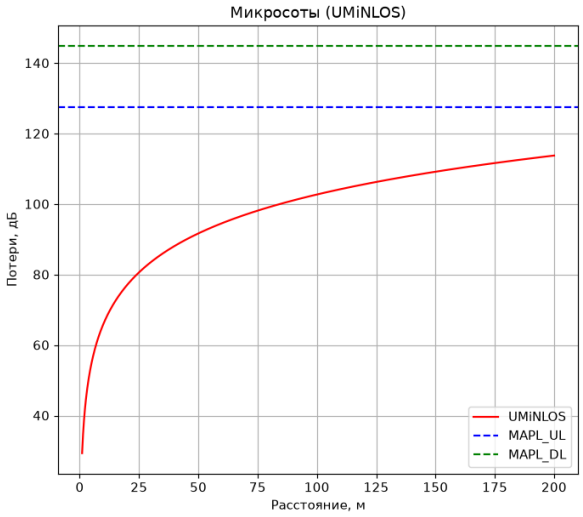
plt.subplot(1, 2, 1)
plt.plot(d_m, PL_UMi, label='UMiNLOS', color='red')
plt.axhline(MAPL_UL, color='blue', linestyle='--', label='MAPL_UL')
plt.axhline(MAPL_DL, color='green', linestyle='--', label='MAPL_DL')
plt.title('Микросоты (UMiNLOS)')
plt.xlabel('Расстояние, м')
plt.ylabel('Потери, дБ')
plt.legend()
plt.grid()

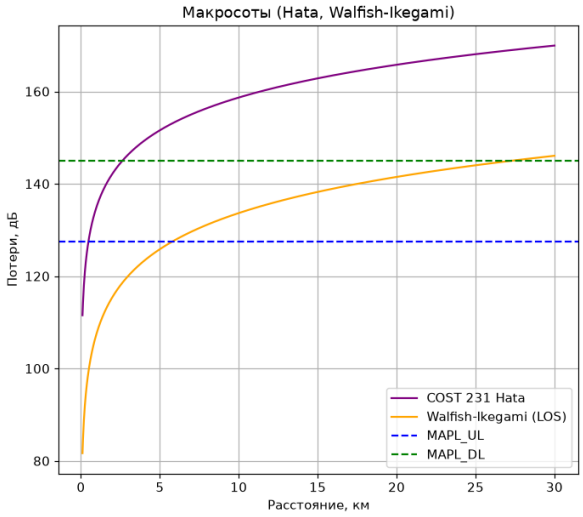
plt.subplot(1, 2, 2)
```

```
plt.plot(d_km, PL_Hata, label='COST 231 Hata', color='purple')
plt.plot(d_km, PL_Walfish, label='Walfish-Ikegami (LOS)', color='orange')
plt.axhline(MAPL_UL, color='blue', linestyle='--', label='MAPL_UL')
plt.axhline(MAPL_DL, color='green', linestyle='--', label='MAPL_DL')
plt.title('Макросоты (Hata, Walfish-Ikegami)')
plt.xlabel('Расстояние, км')
plt.ylabel('Потери, дБ')
plt.legend()
plt.grid()

plt.tight_layout()
plt.show()
```

График:




**4)Определите радиус базовой станции в восходящем и нисходящем каналах. По меньшему из полученных значений рассчитайте площадь одной базовой станции и, исходя из заданной площади, вычислите требуемое количество базовых станций (сайтов), необходимое для обеспечения непрерывного покрытия на этой территории.**

Код:

```
import numpy as np
import matplotlib.pyplot as plt

TxPowerBS = 46
TxPowerUE = 24
AntGainBS = 21
PenetrationM = 15
MIMOGain = 3
IM = 1
FeederLoss = 2
```

```
NoiseFigureBS = 2.4
NoiseFigureUE = 6
ThermalNoiseUL = -174 + 10 * np.log10(10e6)
ThermalNoiseDL = -174 + 10 * np.log10(20e6)
ReqiredSINR_UL = 4
ReqiredSINR_DL = 2

RxSensBS = NoiseFigureBS + ThermalNoiseUL + ReqiredSINR_UL
RxSensUE = NoiseFigureUE + ThermalNoiseDL + ReqiredSINR_DL

MAPL_UL = (TxPowerUE - FeederLoss + AntGainBS + MIMOGain - IM - PenetrationM) - RxSensBS
MAPL_DL = (TxPowerBS - FeederLoss + AntGainBS + MIMOGain - IM - PenetrationM) - RxSensUE

print(f"MAPL_UL = {MAPL_UL} дБ")
print(f"MAPL_DL = {MAPL_DL} дБ")

def UMiNLOS(d_m):
PL = 26 * np.log10(1.8) + 22.7 + 36.7 * np.log10(d_m)
return PL

def COST231_Hata(d_km):
h_bs=30
h_ms=1.5
f_mhz=1800
A = 46.3
B = 33.9
a_hms = (1.1 * np.log10(f_mhz)) * h_ms - (1.56 * np.log10(f_mhz) - 0.8)
L_clutter = 0
s = 44.9 - 6.55 * np.log10(f_mhz)
PL = A + B * np.log10(f_mhz) - 13.82 * np.log10(h_bs) - a_hms + s * np.log10(d_km) + L_clutter
return PL

def Walfish_Ikegami_LOS(d_km):
f_mhz=1800
PL = 42.6 + 20 * np.log10(f_mhz) + 26 * np.log10(d_km)
return PL

d_m = np.linspace(1, 200, 1000)
PL_UMi = UMiNLOS(d_m)

d_km = np.linspace(0.1, 30, 1000)
PL_Hata = COST231_Hata(d_km)
PL_Walfish = Walfish_Ikegami_LOS(d_km)

plt.subplot(1, 2, 1)
plt.plot(d_m, PL_UMi, label='UMiNLOS', color='red')
plt.axhline(MAPL_UL, color='blue', linestyle='--', label='MAPL_UL')
plt.axhline(MAPL_DL, color='green', linestyle='--', label='MAPL_DL')
plt.title('Микросоты (UMiNLOS)')
plt.xlabel('Расстояние, м')
```

```
plt.ylabel('Потери, дБ')
plt.legend()
plt.grid()

plt.subplot(1, 2, 2)
plt.plot(d_km, PL_Hata, label='COST 231 Hata', color='purple')
plt.plot(d_km, PL_Walfish, label='Walfish-Ikegami (LOS)', color='orange')
plt.axhline(MAPL_UL, color='blue', linestyle='--', label='MAPL_UL')
plt.axhline(MAPL_DL, color='green', linestyle='--', label='MAPL_DL')
plt.title('Макросоты (Hata, Walfish-Ikegami)')
plt.xlabel('Расстояние, км')
plt.ylabel('Потери, дБ')
plt.legend()
plt.grid()

plt.tight_layout()
plt.show()

print(f"\nРадиус в UL: 0.47 км")
print(f"Радиус в DL: 2.6 км")
print(f"Итоговый радиус: 0.47 км")

S_bs = 1.95 * (0.47 ** 2)
N_bs = np.ceil(100 / S_bs)

print(f"Площадь одной БС: {S_bs:.2f} кв. км")
print(f"Количество БС на 100 кв. км: {int(N_bs)} шт.")
```

Вывод:

```
Радиус в UL: 0.47 км
Радиус в DL: 2.6 км
Итоговый радиус: 0.47 км
Площадь одной БС: 0.43 кв. км
Количество БС на 100 кв. км: 233 шт.
```

**Вывод:**
Рассчитаны бюджеты каналов UL и DL, найдены максимально допустимые потери (MAPL_UL = **127.6** дБ, MAPL_DL = **144.9897000433602** дБ). По модели COST 231 Hata определён радиус соты — **0.47** км. Площадь одной трёхсекторной БС составила 0.43 кв. км. Для покрытия 100 кв. км необходимо **233** базовых станций.

Ссылка на Git:

Ответы на контрольные вопросы:

1) Для чего используются прямое и обратное преобразование Фурье?
2) Что такое ошибка квантования и дискретизации?
3) Какое количество разрядов АЦП требуется, чтобы оцифровать голос?
16
4) Как математически получить дискретные отсчеты непрерывного сигнала?
5) Какой спектр у периодического сигнала $\sin(10\pi t+\pi/2)$?
6) Что такое быстрое преобразование Фурье?
7) Как определяется минимальная требуемая для оцифровки частота дискретизации сигнала?

1)
Прямое - переводит сигнал из времени в частоту (получить спектр). Обратное - из частоты обратно во время (восстановить сигнал по спектру).

2)
Квантование - разница между реальным значением и округлённым до уровня (шум квантования). Дискретизация - замена непрерывного сигнала отсчётами через интервал; ошибка - потеря информации между отсчётами (наложение спектров, если частота мала).

3)
Обычно 16 бит (диапазон ~96 дБ, стандарт для речи/аудио). Минимум - 12–14 бит.

4)
Умножением непрерывного сигнала на гребёнку Дирака:
$x_d[n] = x(nT)$, где $T = 1/f_д$ - период дискретизации.

5)
Одна гармоника на частоте $f = 5$ Гц (т.к. $\omega = 10\pi$). Спектр - две дельта-функции на ±5 Гц (или одна линия на 5 Гц в одностороннем виде). Амплитуда 1, фаза $\pi/2$.

6)
Алгоритм, вычисляющий дискретное преобразование Фурье за $N \cdot \log_2 N$ операций вместо $N^2$. Просто ускоренный способ посчитать спектр.

7)

По теореме Котельникова: $f_д \geq 2 \cdot f_{max}$, где $f_{max}$ — максимальная частота в спектре сигнала.